

Trabajo web 1

TP 1: Desarrollo de la propuesta por escrito en un proyecto

El desarrollo del proyecto es una propuesta de trabajo que debe incluir obligatoriamente:

1.1. Equipo y roles (salvo que se trabaje solo o sola) que pueden ir rotando.

El equipo está formado por **Mateo Suppo** encargado del Desarrollo Frontend, **Sofia Chimento** se encarga del Diseño y Gestora de Proyecto (Project Manager)

1.2 Lo que se va a desarrollar y su justificación (¿por qué es necesario y qué problema resuelve o qué propuesta de valor brinda?).

Se desarrolla la página para que el cliente tenga una web donde puedan hacer pedidos de sus productos y turnos para peluquería que se ofrecen en este PetShop. Le ayuda a gestionar los turnos de peluquería y el stock de insumos. Agiliza la toma de turnos en la peluquería y facilita la compra de los productos

1.3 Cronograma de avances y presentaciones. Un documento que dé cuenta de las fechas (deadline) para las entregas. Utilización de una herramienta de organización de tareas.

Utilizaremos trello por el hecho que ya lo hemos usado en otra ocasión

link:

<https://trello.com/invite/b/6a8f56a3339276d077bc572d/ATTI9470d3813431cc3074bf0adf23cf38dbD9584412/huellaycia>

Link de repositorio:

<https://github.com/msuppocastillo-design/huellasycia.git>

TP 2: Back-end para servir el sitio web

1 Lo que hicimos fue agregarle un servidor a nuestra página que habíamos armado en el TP1. La idea principal era dejar de abrir la página haciendo doble clic en los archivos sueltos y pasar a levantarla usando Node.js con Express. Además, sacamos todos los productos que estaban escritos a mano dentro del código de JavaScript y armamos todo para que la página los traiga de internet desde una API, como hacen las páginas de verdad.

2. Herramientas que usamos

- Node.js: Para poder correr el servidor en la compu.
- Express: Una herramienta de Node que nos ayuda a manejar las URLs de la página y mostrar los archivos.

- **MockAPI:** Una página de internet que usamos para simular una base de datos y guardar ahí nuestros productos.
- **Fetch:** La función que usamos para pedir los datos de los productos y traerlos al servidor.

3. Cómo organizamos los archivos Tratamos de separar el código en distintas carpetas para que no quede todo mezclado y sea más fácil de entender si hay que arreglar algo:

- **index.mjs:** Es el archivo principal, el que arranca el servidor.
- **src/routers:** Acá armamos las rutas (las direcciones) para buscar los productos.
- **src/controllers:** Acá está el código que recibe el pedido del usuario y manda la respuesta.
- **src/services:** Esta parte es la que se conecta directamente con MockAPI para pedir los datos.
- **public:** Acá metimos todo el código viejo del TP1 (el HTML, CSS, imágenes y el JS de la página).

Hacerlo así sirve para que, si el día de mañana cambiamos MockAPI por una base de datos real, solo tengamos que cambiar la carpeta de servicios y no tocar el resto del servidor.

4. Conexión con los datos (MockAPI) Usamos la página MockAPI.io para guardar los 14 productos que teníamos en el catálogo. Le cargamos los mismos datos que antes (nombre, precio, foto, descripción, etc.) y la página le genera un ID automático a cada uno.

Nuestro servidor funciona como un puente: la página web (el cliente) nunca le pide los productos directo a MockAPI. La página nos los pide a nosotros (a **/api/productos**), nuestro servidor va, los busca en MockAPI, y se los devuelve a la página.

Para que esto ande, tuvimos que cambiar el archivo **funciones.js** de la página para que deje de leer los productos de un arreglo fijo y empiece a usar **fetch** para traerlos cada vez que alguien entra a la tienda.

5. Las rutas que armamos

- **[GET] /api/productos:** Sirve para traer la lista completa. Busca todos los productos en MockAPI y te los devuelve listos para mostrar.
- **[GET] /api/productos/:id:** Sirve para buscar un solo producto usando su número de ID (por ejemplo, para cuando el usuario hace clic para ver el detalle de un producto). Si el ID no existe, devuelve un error 404 diciendo que no lo encontró.

6. Conclusión Con este TP logramos que la página deje de ser una página estática básica y pase a tener su propio servidor funcionando. Haber separado todo en carpetas ordenadas nos va a venir muy bien para el futuro, por si le queremos agregar más cosas a la página.